SC2006 LAB 4

# Software Testing Report

Black-box equivalence-class and boundary-value testing, plus white-box basis-path testing

| | |
|---|---|
| Work product | Test Cases and Testing Results |
| Prepared by | Group 1 - Software Testing |
| System | Group Scheduling Application |
| Target code | feature/production-pilot tree at 26cb622 |
| Execution date | 7 November 2026 (Asia/Singapore) |
| Result | 29/29 assessment tests passed |

## Submission scope

This report satisfies Lab 4 Manual sections 3.2.1-3.2.4. It tests one important control boundary using equivalence classes and boundary values, tests two complex methods using basis paths, minimises redundant inputs, and records input, expected output, actual output, and pass/fail results.

# 1 Executive Summary

Black-box testing treats the recommendation control boundary as a sealed unit: tests provide complete recommendation requests and inspect only public results or validation errors. White-box testing reads the method structure, draws its control-flow graph, calculates cyclomatic complexity, and executes a basis set of independent paths.

**All 29 assessment-focused automated tests passed. The complete backend regression suite passed 293 tests; all 21 frontend tests, lint, and the production build also passed. No production behaviour was changed for this work product.**

## Selected test subjects

| Technique | Selected unit | Reason for selection |
|---|---|---|
| Black-box | RecommendationEngine.generate() | Core UC-09/UC-10 control boundary; validates the request and coordinates timeslots, venues, routing, feasibility, scoring, and ranking. |
| White-box 1 | pre_routing_exclusion() | Implements FR-D1, FR-D4, and FR-D5 with ordered feasibility decisions before external routing. |
| White-box 2 | _ParticipantAvailability.status_for() | Implements four distinct attendance states and the BUSY-overrides-AVAILABLE rule. |

## Environment and evidence

| Item | Value |
|---|---|
| Language/runtime | Python 3.13.5 |
| Test framework | pytest 8.3.5 |
| Assessment run | 29 passed, 0 failed, 0 errors |
| Full backend regression | 293 passed; one dependency deprecation warning |
| Frontend verification | 21 tests passed; lint passed; production build passed |
| External routing | Deterministic test double at provider port; all application logic remains real |

# 2 Black-box Test Design

The control boundary is tested without using its internal branch structure. Inputs were partitioned by externally observable behaviour. Boundary values use the nearest invalid value and each valid endpoint where numeric limits exist.

## 2.1 Equivalence classes

| Input characteristic | Equivalence classes | Representative cases |
|---|---|---|
| Session duration | Invalid low: <15; valid: 15-480; invalid high: >480 | BB-DUR-01..04 |
| Attendance outcome | Threshold met; requested threshold missed but best>=2; no two-person group | BB-THR-01..03 |
| Venue compatibility | At least one supporting venue; no supporting venue | BB-THR-01, BB-VEN-01 |
| Participant locations | At least one active valid origin; no valid origin | BB-THR-01, BB-LOC-01 |
| Routing provider | Available duration; explicitly unavailable; exception/error | BB-JRN-01, BB-RTE-01..02 |
| Individual journey | At or below 60-minute cap; above cap | BB-JRN-01..02 |
| Attendance weight | Invalid low: <0; valid: 0-100; invalid high: >100 | BB-WGT-01..04 |

## 2.2 Boundary-value analysis

| Boundary | Values selected | Expected behaviour | Cases |
|---|---|---|---|
| Duration minimum 15 | 14, 15 | 14 rejected; 15 accepted | BB-DUR-01..02 |
| Duration maximum 480 | 480, 481 | 480 accepted; 481 rejected | BB-DUR-03..04 |
| Weight minimum 0 | -1, 0 | -1 rejected; 0 accepted | BB-WGT-03, BB-WGT-01 |
| Weight maximum 100 | 100, 101 | 100 accepted; 101 rejected | BB-WGT-02, BB-WGT-04 |
| Journey cap 60 | 60, 61 | 60 retained; 61 excluded | BB-JRN-01..02 |

## 2.3 Minimisation rationale

The test set uses one representative for each non-boundary partition and the adjacent values around every numeric boundary. Cases deliberately cover several compatible conditions at once. For example, BB-THR-01 also supplies a valid venue, valid locations, successful routing, and a valid mid-range weight. Additional combinations that would repeat the same observable class were omitted.

# 3 Black-box Test Cases and Results

| Case | Req. | Test input / precondition | Expected output | Actual output | Result |
|---|---|---|---|---|---|
| BB-DUR-01 | FR-A1.1 | duration=14 min | Reject below 15 | ValueError: duration must be 15 to 480 minutes (FR-A1.1) | PASS |
| BB-DUR-02 | FR-A1.1 | duration=15 min | Accept minimum | RunStatus.OK; ranked options returned | PASS |
| BB-DUR-03 | FR-A1.1 | duration=480 min | Accept maximum | RunStatus.OK; ranked options returned | PASS |
| BB-DUR-04 | FR-A1.1 | duration=481 min | Reject above 480 | ValueError: duration must be 15 to 480 minutes (FR-A1.1) | PASS |
| BB-THR-01 | FR-D1 / UC-09 | 2 overlap; threshold=2 | OK with two attendees | RunStatus.OK; all ranked options have attendee_count=2 | PASS |
| BB-THR-02 | UC-09 recovery | best=2; requested=3 | Label best-available fallback | BEST_AVAILABLE_BELOW_THRESHOLD; two-attendee options; warning names 3 | PASS |
| BB-THR-03 | FR-D1 / UC-09 | no overlap of 2; threshold=2 | No eligible timeslot; no routing | NO_TIMESLOT_MEETS_THRESHOLD; 0 combinations; 0 routing calls | PASS |
| BB-VEN-01 | FR-C3 | activity=badminton; venue=football | No supporting venue | NO_SUPPORTING_VENUES; no ranked options | PASS |
| BB-LOC-01 | FR-C4 / NFR-3 | all locations invalid/unavailable | No centroid; no routing | NO_VALID_LOCATIONS; no ranked options; 0 routing calls | PASS |
| BB-RTE-01 | NFR-3 / NFR-8 | provider UNAVAILABLE; minutes=None | Keep data unavailable; never invent 0 | All combinations DATA_UNAVAILABLE; all minutes=None; no ranking | PASS |
| BB-RTE-02 | NFR-3 | provider raises RuntimeError | Contain error and continue safely | All travel statuses ERROR; all combinations DATA_UNAVAILABLE | PASS |
| BB-JRN-01 | FR-D3 | journey=60 min; cap=60 | Keep boundary value | RunStatus.OK; ranked options; limit flags False | PASS |
| BB-JRN-02 | FR-D3 | journey=61 min; cap=60 | Exclude above cap | No ranked options; every reason JOURNEY_LIMIT | PASS |
| BB-WGT-01 | FR-D9 | attendance weight=0% | Accept minimum | RunStatus.OK; reported weight=0; ranked options returned | PASS |
| BB-WGT-02 | FR-D9 | attendance weight=100% | Accept maximum | RunStatus.OK; reported weight=100; ranked options returned | PASS |
| BB-WGT-03 | FR-D9 | attendance weight=-1% | Reject below 0 | ValueError: attendance weight must be between 0 and 100 | PASS |
| BB-WGT-04 | FR-D9 | attendance weight=101% | Reject above 100 | ValueError: attendance weight must be between 0 and 100 | PASS |

**Evidence: every row above corresponds to one passing pytest function with the same case ID. The JUnit result contained 17 black-box cases, 0 failures, and 0 errors.**

# 4 White-box Method 1: Pre-routing Exclusion

This method applies feasibility rules in priority order before any routing request: the session threshold (FR-D1), sports-facility minimum group size (FR-D4), and sports-facility booking lead time (FR-D5). The compound minimum-group expression is split into two short-circuit decisions in the control-flow graph.

## 4.1 Source excerpt

```
20  def pre_routing_exclusion(
21      combo: Combination,
22      minimum_attendee_threshold: int,
23      now: datetime,
24  ) -> ExclusionReason | None:
25      """FR-D1, FR-D4 and FR-D5, in that order. Returns the first reason that applies."""
26      count = combo.attendee_count
27      venue = combo.venue
28
29      # FR-D1: attendance below the Session's Minimum-Attendee Threshold.
30      if count < minimum_attendee_threshold:
31          return ExclusionReason.BELOW_THRESHOLD
32
33      if venue.category is VenueCategory.SPORTS_FACILITY:
34          # FR-D4: attendance below the Venue's minimum group size.
35          if venue.min_group_size is not None and count < venue.min_group_size:
36              return ExclusionReason.GROUP_SIZE
37          # FR-D5: Timeslot falls inside the Venue's required advance-booking window,
38          # i.e. it starts sooner than the Venue's minimum booking lead time.
39          if venue.advance_booking_lead_minutes:
40              earliest_bookable = now + timedelta(minutes=venue.advance_booking_lead_minutes)
41              if combo.timeslot.start < earliest_bookable:
42                  return ExclusionReason.BOOKING_LEAD_TIME
43
44      return None
```

## 4.2 Control-flow graph

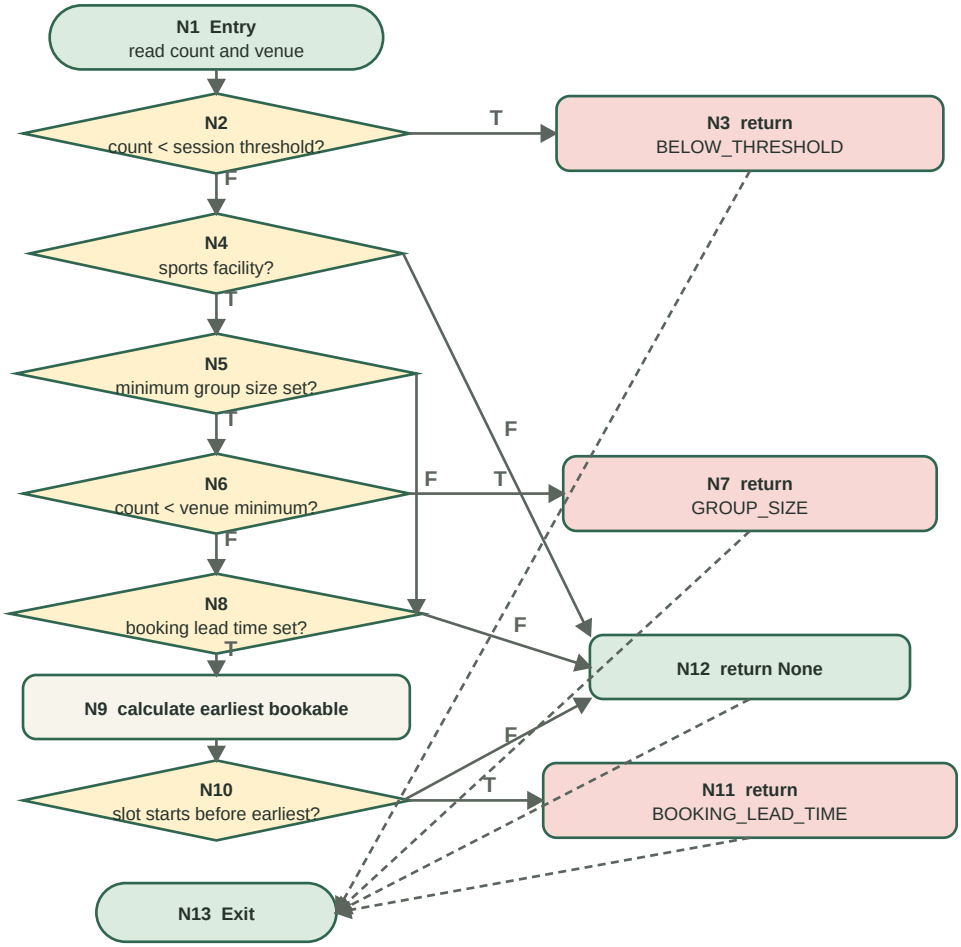


Figure 1. Control-flow graph for pre_routing_exclusion(). Node numbering is identical to the DOT source committed with the tests.

### 4.3 Cyclomatic complexity

| Calculation | Value |
|---|---|
| Predicate method | 6 decision nodes + 1 = 7 |
| Graph method | E - N + 2 = 18 - 13 + 2 = 7 |
| Required independent paths | 7 |

### 4.4 Independent paths and executed results

| Case | Independent node path | Input | Expected | Actual | Result |
|---|---|---|---|---|---|
| WB-PR-01 | N1-N2(T)-N3-N13 | count=2, threshold=3 | BELOW_THRESHOLD | BELOW_THRESHOLD | PASS |
| WB-PR-02 | N1-N2(F)-N4(F)-N12-N13 | park; count=threshold=3 | None | None | PASS |
| WB-PR-03 | N1-N2(F)-N4(T)-N5(T)-N6(T)-N7-N13 | sports; count=3; venue min=4 | GROUP_SIZE | GROUP_SIZE | PASS |
| WB-PR-04 | N1-N2(F)-N4(T)-N5(F)-N8(F)-N12-N13 | sports; no min; no lead | None | None | PASS |
| WB-PR-05 | N1-N2(F)-N4(T)-N5(T)-N6(F)-N8(F)-N12-N13 | sports; count=min=4; no lead | None | None | PASS |
| WB-PR-06 | N1-N2(F)-N4(T)-N5(T)-N6(F)-N8(T)-N9-N10(T)-N11-N13 | 24h lead; start only 23h away | BOOKING_LEAD_TIME | BOOKING_LEAD_TIME | PASS |
| WB-PR-07 | N1-N2(F)-N4(T)-N5(T)-N6(F)-N8(T)-N9-N10(F)-N12-N13 | 24h lead; start exactly 24h away | None | None | PASS |

# 5 White-box Method 2: Participant Attendance Status

This method preserves four distinct participant states. A missing response and a withdrawn response are returned before interval processing. For an active response, any BUSY overlap overrides AVAILABLE time; otherwise the slot must be fully contained in an AVAILABLE block.

### 5.1 Source excerpt

```
111      def status_for(self, start: datetime, end: datetime) -> AttendanceStatus:
112          p = self.participant
113          if p.record_status is None:
114              return AttendanceStatus.NO_RESPONSE
115          if p.record_status is RecordStatus.WITHDRAWN:
116              return AttendanceStatus.WITHDRAWN
117          # A BUSY block (e.g. imported from Google Calendar, FR-B8) that overlaps
118          # any part of the slot makes the Participant unavailable for it.
119          if any(bs < end and start < be for bs, be in self.busy):
120              return AttendanceStatus.UNAVAILABLE
121          # The slot must lie wholly inside one merged AVAILABLE block.
122          if any(as_ <= start and end <= ae for as_, ae in self.available):
123              return AttendanceStatus.AVAILABLE
124          return AttendanceStatus.UNAVAILABLE
```

## 5.2 Control-flow graph

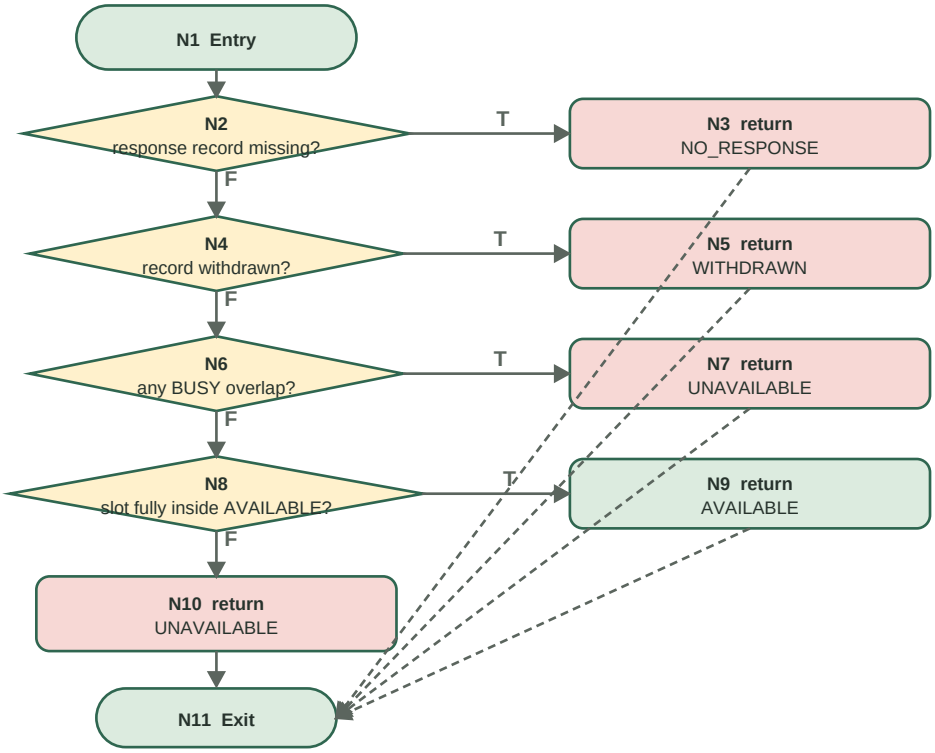


Figure 2. Application-level CFG for _ParticipantAvailability.status_for(). Each any(...) result is one boolean decision; Python iterator internals are outside this graph's abstraction boundary.

## 5.3 Cyclomatic complexity

| Calculation | Value |
|---|---|
| Predicate method | 4 decision nodes + 1 = 5 |
| Graph method | E - N + 2 = 14 - 11 + 2 = 5 |
| Required independent paths | 5 |

## 5.4 Independent paths and executed results

| Case | Independent node path | Input | Expected | Actual | Result |
|---|---|---|---|---|---|
| WB-PA-01 | N1-N2(T)-N3-N11 | record_status=None | NO_RESPONSE | NO_RESPONSE | PASS |
| WB-PA-02 | N1-N2(F)-N4(T)-N5-N11 | record_status=WITHDRAWN | WITHDRAWN | WITHDRAWN | PASS |
| WB-PA-03 | N1-N2(F)-N4(F)-N6(T)-N7-N11 | available covers slot; BUSY overlaps | UNAVAILABLE | UNAVAILABLE | PASS |
| WB-PA-04 | N1-N2(F)-N4(F)-N6(F)-N8(T)-N9-N11 | slot fully inside AVAILABLE | AVAILABLE | AVAILABLE | PASS |
| WB-PA-05 | N1-N2(F)-N4(F)-N6(F)-N8(F)-N10-N11 | AVAILABLE ends 1 min early | UNAVAILABLE | UNAVAILABLE | PASS |

# 6 Execution Evidence and Cross-check

| Verification | Observed result | Status |
|---|---|---|
| Lab 4 black-box tests | 17 passed, 0 failed, 0 errors | PASS |
| Lab 4 white-box tests | 12 passed, 0 failed, 0 errors | PASS |
| Combined Lab 4 JUnit run | 29 passed, 0 failed, 0 errors | PASS |
| Full backend regression | 293 passed; 1 third-party deprecation warning | PASS |
| Frontend unit/UI tests | 21 passed | PASS |
| Frontend lint | oxlint exit code 0 | PASS |
| Frontend production build | 152 modules transformed; build exit code 0 | PASS |

## 6.1 Manual compliance matrix

| Manual requirement | Evidence in this report | Status |
|---|---|---|
| 3.2.1: one important control class | RecommendationEngine.generate(); Sections 2-3 | Met |
| 3.2.1: equivalence classes | Section 2.1 and BB test rows | Met |
| 3.2.1: boundary values | Section 2.2; 14/15, 480/481, -1/0, 100/101, 60/61 | Met |
| 3.2.2: two complex methods | Sections 4 and 5 | Met |
| 3.2.2: basis-path testing | Two CFGs, two complexity calculations, 12 path cases | Met |
| 3.2.3: minimise cases | One representative per partition plus boundary neighbours; Section 2.3 | Met |
| 3.2.4: execute and document | Every row has input, expected, actual, and result; Section 6 | Met |

## 6.2 Consistency checks

1. Each of the 29 report case IDs maps one-to-one to a unique JUnit test name.
2. Every equivalence class and boundary in Section 2 maps to at least one executed case.
3. The first CFG has N=13 and E=18; both formulas give V(G)=7; seven independent paths were executed.
4. The second CFG has N=11 and E=14; both formulas give V(G)=5; five independent paths were executed.
5. Actual outputs are literal observations asserted by the final passing tests, not restatements of expected output.
6. No secrets, access tokens, credentials, or personal availability data appear in this report.

# 7 Limitations and Conclusion

## 7.1 Limitations

The black-box control tests use a deterministic router at the external provider port so that boundary outcomes are reproducible and do not depend on OneMap availability. This is appropriate for unit-level control-class testing but does not replace the separate deployed smoke test of Google authentication, OneMap, PostgreSQL, and Render. The report also uses an application-level CFG: the internal iteration performed by Python's any(...) is treated as part of expression evaluation rather than as a separate business-logic decision.

## 7.2 Conclusion

**The selected tests provide systematic evidence for the recommendation control logic and two complex decision methods. Equivalence partitioning, boundary-value analysis, and basis-path testing were applied consistently; redundant combinations were avoided; all expected and actual results are traceable to automated test IDs; and the full regression checks found no introduced failures.**

**Reference.** SC2006 Lab 4 Manual: Implementation, Testing, and Preparation for Demo, sections 3.2.1-3.2.4. Source code and automated tests are available on branch lab4/software-testing-report.